Testing in Go - Cheat Sheet

Go ships with a testing framework in the standard library. No external framework required. Tests live in `_test.go` files, look like regular Go code, and run with `go test`. The same toolchain handles benchmarks, examples, fuzz tests, and coverage.

The opinion is “tests are code”. You write them like you write everything else. No annotations, no DSL, no XML.

File and function layout

Test files end in `_test.go`. They live in the same package as the code they test (or a sibling `_test` package for black-box tests).

```
calc/
    calc.go            // package calc
    calc_test.go       // package calc (white-box test)
    calc_ext_test.go   // package calc_test (black-box test)
```

A test function starts with `Test`, takes `*testing.T`, returns nothing.

```go
package calc

import "testing"

func TestAdd(t *testing.T) {
    got := Add(2, 3)
    want := 5
    if got != want {
        t.Errorf("Add(2, 3) = %d, want %d", got, want)
    }
}
```

Run it:

```
go test                          # current package
go test ./...                    # entire module
go test ./calc                   # one package
go test -v                       # verbose
go test -run TestAdd             # only matching tests
go test -run TestAdd/edge_case   # specific subtest
go test -count=3                 # run each test N times (defeats caching)
```

The *testing.T object

Every test gets a `*testing.T`. It's how you report failures, log, skip, mark parallel, and register cleanup.

Method	What it does
`t.Error / t.Errorf`	Report a failure, keep running
`t.Fatal / t.Fatalf`	Report a failure, stop the test
`t.Log / t.Logf`	Log a line (only shown with `-v` or on failure)
`t.Skip / t.Skipf`	Skip the rest of the test
`t.SkipNow`	Skip immediately
`t.Helper`	Mark the function as a test helper
`t.Cleanup(fn)`	Register a function to run when the test ends
`t.TempDir`	Create a temporary directory, auto-cleaned
`t.Setenv`	Set an env var, auto-restored
`t.Parallel`	Mark the test as parallel-safe
`t.Run`	Run a subtest
`t.Failed`	Did the test fail so far?

The standard pattern uses `Error` (keep going) or `Fatal` (stop here):

```
if err != nil {
    t.Fatalf("unexpected error: %v", err)
}
```

Use `Fatal` when subsequent assertions would crash or be misleading. Use `Error` when you can collect more useful failures by continuing.

---

t.Helper

Marks a function so its frame is skipped in failure reports. Errors point at the caller, where they're useful.

```go
func mustParse(t *testing.T, s string) int {
    t.Helper()
    n, err := strconv.Atoi(s)
    if err != nil {
        t.Fatalf("parse %q: %v", s, err)
    }
    return n
}

func TestSomething(t *testing.T) {
    n := mustParse(t, "42")  // failure shows this line, not mustParse internals
    _ = n
}
```

Any function that calls `t.Error` or `t.Fatal` on behalf of the test should mark itself as a helper.

Table-driven tests

The idiomatic Go pattern for testing many inputs.

```go
func TestAdd(t *testing.T) {
    cases := []struct {
        name string
        a, b int
        want int
    }{
        {"positive", 2, 3, 5},
        {"zero", 0, 0, 0},
        {"negative", -1, -2, -3},
        {"mixed", -5, 5, 0},
    }

    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.want {
                t.Errorf("Add(%d, %d) = %d, want %d", tc.a, tc.b, got, tc.want)
            }
        })
    }
}
```

`t.Run` creates a subtest. Subtest names show up in output and let you filter:

```
go test -run "TestAdd/positive"
go test -run "TestAdd/(positive|zero)"
```

Subtests

`t.Run` makes a child test. Used for table tests, scoping setup, or just organizing related assertions.

```
func TestUser(t *testing.T) {
    u := newUser()

    t.Run("default name", func(t *testing.T) {
        if u.Name != "" {
            t.Errorf("got %q, want empty", u.Name)
        }
    })

    t.Run("set name", func(t *testing.T) {
        u.SetName("alice")
        if u.Name != "alice" {
            t.Errorf("got %q, want %q", u.Name, "alice")
        }
    })
}
```

A subtest's failure doesn't stop sibling subtests (unlike `Fatal` in a flat test).

Parallel tests

`t.Parallel()` marks the test as safe to run alongside other parallel tests. The runner schedules them across `GOMAXPROCS` goroutines.

```
func TestSomething(t *testing.T) {
    t.Parallel()

    cases := []struct{ /* ... */ }{ /* ... */ }
    for _, tc := range cases {
        tc := tc  // capture for closure (pre-Go 1.22)
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel()
            // test body
        })
    }
}
```

Pre-Go 1.22, you needed `tc := tc` because the loop variable was shared. From 1.22+, each iteration has its own `tc`.

Parallel tests must not share mutable state. Use `t.TempDir`, fresh objects per test, and avoid global state.

Setup and teardown

t.Cleanup

The modern way. Register a function that runs when the test (or subtest) ends.

```
func TestWithFile(t *testing.T) {
    f, err := os.CreateTemp("", "test")
    if err != nil {
        t.Fatal(err)
    }
    t.Cleanup(func() {
        os.Remove(f.Name())
    })

    // use f
}
```

`t.Cleanup` runs at the end of the test (or subtest), in LIFO order across multiple calls. It also runs after `t.Fatal`. Useful when a helper needs to register cleanup that fires at the test boundary, not at the helper's return.

t.TempDir and t.Setenv

Two helpers that handle their own cleanup:

```
func TestDB(t *testing.T) {
    dir := t.TempDir()                      // auto-removed at end of test
    t.Setenv("DATABASE_PATH", dir+"/db")    // restored at end of test
    // ...
}
```

TestMain

For setup that runs once across all tests in a package:

```
func TestMain(m *testing.M) {
    // setup
    setupDatabase()

    code := m.Run()      // run all tests

    // teardown
    cleanupDatabase()

    os.Exit(code)
}
```

Only one per package. Useful for spinning up containers (Testcontainers-go), seeding test data, or any expensive setup.

Benchmarks

A function starting with `Benchmark` and taking `*testing.B`.

```
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(2, 3)
    }
}
```

Run:

```
go test -bench .                # all benchmarks in current package
go test -bench BenchmarkAdd
go test -bench . -benchtime=5s  # run each benchmark for 5 seconds
go test -bench . -benchmem      # report allocations
go test -bench . -cpu 1,2,4,8   # vary GOMAXPROCS
```

The runner picks `b.N` to make the benchmark run long enough for stable numbers. Your code calls `Add` `b.N` times per iteration.

Useful B methods

```
func BenchmarkSomething(b *testing.B) {
    // Expensive setup
    data := buildLargeData()
    b.ResetTimer()       // start measuring after setup

    for i := 0; i < b.N; i++ {
        process(data)
    }

    b.StopTimer()        // pause for teardown
    cleanup(data)
}

func BenchmarkWithAlloc(b *testing.B) {
    b.ReportAllocs()     // include allocs/op in output

    for i := 0; i < b.N; i++ {
        _ = make([]int, 1000)
    }
}
```

Output:

```
BenchmarkAdd-8       1000000000     0.3 ns/op    0 B/op    0 allocs/op
```

That's: name, GOMAXPROCS, iterations, time per op, bytes per op, allocs per op.

benchstat

Compare two benchmark runs:

```
go test -bench . -count=10 > old.txt
# make changes
go test -bench . -count=10 > new.txt

benchstat old.txt new.txt
```

`-count=10` runs each benchmark 10 times for statistical significance.

Examples

Functions starting with `Example` serve double duty: they run as tests and appear in `go doc`.

```
func ExampleAdd() {
    fmt.Println(Add(2, 3))
    // Output: 5
}
```

The `// Output:` comment tells the test what to expect on stdout. If the output doesn't match, the test fails.

Examples for specific identifiers:

```
func ExampleParser_Parse() { /* ... */ }      // method on Parser
func ExampleAdd_negative() { /* ... */ }       // variation
```

Useful for keeping docs honest. Examples that don't compile or produce wrong output break the build.

Fuzz tests (Go 1.18+)

Generate random inputs and try to make the code crash or produce inconsistent results.

```go
func FuzzReverse(f *testing.F) {
    f.Add("hello")            // seed corpus
    f.Add("")
    f.Add("Privet")

    f.Fuzz(func(t *testing.T, s string) {
        reversed := Reverse(s)
        doubleReversed := Reverse(reversed)
        if s != doubleReversed {
            t.Errorf("reverse twice changed %q to %q", s, doubleReversed)
        }
        if !utf8.ValidString(reversed) {
            t.Errorf("reverse produced invalid UTF-8: %q", reversed)
        }
    })
}
```

Run:

```
go test -fuzz FuzzReverse -fuzztime 30s
```

Failing inputs get saved to `testdata/fuzz/FuzzReverse/...` and become permanent regression tests.

Test coverage

Built into `go test`. No extra tools.

```
go test -cover                  # print coverage summary
go test -coverprofile=cover.out  # write profile to file
go tool cover -html=cover.out    # browse coverage as HTML
go tool cover -func=cover.out    # per-function summary
```

Three coverage modes:

- `-covermode=set` (default): did this statement run? (boolean)
- `-covermode=count`: how many times did each statement run?
- `-covermode=atomic`: same as count but safe under `-race` for concurrent tests.

For multi-package coverage:

```
go test -coverpkg=./... -coverprofile=cover.out ./...
```

Cover-by-default in CI catches gaps. Aim for meaningful tests. 80% coverage with strong assertions does more than 100% with empty asserts.

Race detector

```
go test -race ./...
```

Instruments memory accesses. Reports unsynchronized reads/writes from different goroutines. 5-10x slower at runtime, but worth running on every CI build.

A test that passes under `-race` doesn't prove no race exists. It proves the test didn't observe one. Run race-checked tests with workloads that mirror real concurrent usage.

Test flags reference

```
-v                  # verbose, show passing test names
-run REGEX          # only run matching tests/subtests
-skip REGEX         # skip matching tests
-count N            # run each test N times
-timeout DURATION   # fail tests that take longer (default 10m)
-short              # tests honor testing.Short()
-failfast           # stop on first failure
-cover              # show coverage
-coverprofile FILE  # write coverage profile
-race               # data race detector
-bench REGEX        # run matching benchmarks
-benchmem           # benchmarks report allocations
-benchtime DURATION # benchmark duration per case
-cpu LIST           # GOMAXPROCS values to run with
-parallel N         # max parallel tests (default GOMAXPROCS)
-shuffle on         # randomize test order (Go 1.17+)
```

`testing.Short()` lets a test skip long-running work in CI's fast path:

```
func TestExpensive(t *testing.T) {
    if testing.Short() {
        t.Skip("skipping in short mode")
    }
    // expensive test
}
```

---

testify

The popular third-party assertion library. Four packages:

- **`assert`**: continues on failure (like `t.Error`)

- `require` : stops on failure (like `t.Fatal`)
- `mock` : mocking framework
- `suite` : test suite with setup/teardown

```go
import (
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestUser(t *testing.T) {
    u, err := NewUser("alice")
    require.NoError(t, err)              // stop if error
    assert.Equal(t, "alice", u.Name)     // continue regardless
    assert.True(t, u.Active)
    assert.Len(t, u.Tags, 3)
    assert.Contains(t, u.Tags, "admin")
}
```

testify cuts noise out of common assertions. Style debate: the Go core team prefers the standard `if got != want` form. Both work. Pick a style and stick with it.

Mocks

Hand-rolled mocks via interfaces

The most common style. Define an interface, write a mock that implements it.

```go
type UserStore interface {
    Find(id int) (*User, error)
}

// In tests:
type stubStore struct {
    user *User
    err  error
}

func (s *stubStore) Find(id int) (*User, error) {
    return s.user, s.err
}

func TestService_GetUser(t *testing.T) {
    s := NewService(&stubStore{user: &User{Name: "alice"}})
    got, _ := s.GetUser(42)
    if got.Name != "alice" {
        t.Errorf("got %q, want %q", got.Name, "alice")
    }
}
```

Simple, fast, no codegen.

gomock

Generates mocks from interfaces with expected call recording.

```
mockgen -source=store.go -destination=store_mock.go -package=mypkg
```

```go
ctrl := gomock.NewController(t)
defer ctrl.Finish()

mockStore := NewMockUserStore(ctrl)
mockStore.EXPECT().Find(42).Return(&User{Name: "alice"}, nil)

s := NewService(mockStore)
_, _ = s.GetUser(42)
```

Powerful but verbose. Useful when you have many interfaces or need strict call verification.

Other tools

- **mockery**: similar to gomock, different code style.
- **testify/mock**: lighter alternative bundled with testify.

For most code, hand-rolled stubs are enough. Generated mocks pay off when interfaces have many methods or you need ordered call expectations.

---

httptest

Standard library helpers for testing HTTP code.

Test server

```
import "net/http/httptest"

func TestClient(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        if r.URL.Path != "/users/42" {
            t.Errorf("unexpected path: %s", r.URL.Path)
        }
        w.WriteHeader(200)
        w.Write([]byte(`{"id":42,"name":"alice"}`))
    }))
    t.Cleanup(srv.Close)

    client := NewClient(srv.URL)
    u, err := client.GetUser(42)
    if err != nil {
        t.Fatal(err)
    }
    if u.Name != "alice" {
        t.Errorf("got %q, want %q", u.Name, "alice")
    }
}
```

Recorder for handlers

Test handlers without spinning up a server.

```
func TestHandler(t *testing.T) {
    req := httptest.NewRequest("GET", "/users/42", nil)
    rec := httptest.NewRecorder()

    handler(rec, req)

    if rec.Code != 200 {
        t.Errorf("status = %d, want 200", rec.Code)
    }
    if !strings.Contains(rec.Body.String(), "alice") {
        t.Errorf("body missing 'alice': %q", rec.Body.String())
    }
}
```

Testing time-dependent code

Code that calls `time.Now()` directly is hard to test. Inject a clock.

```
type Clock interface {
    Now() time.Time
}

type realClock struct{}
func (realClock) Now() time.Time { return time.Now() }

type fakeClock struct {
    t time.Time
}
func (f *fakeClock) Now() time.Time          { return f.t }
func (f *fakeClock) Advance(d time.Duration) { f.t = f.t.Add(d) }

type Service struct {
    clock Clock
}

// Tests:
func TestExpiry(t *testing.T) {
    clk := &fakeClock{t: time.Date(2025, 1, 1, 12, 0, 0, 0, time.UTC)}
    s := &Service{clock: clk}

    s.Set("key", "val")
    clk.Advance(2 * time.Hour)
    if v, ok := s.Get("key"); ok {
        t.Errorf("expected expired, got %v", v)
    }
}
```

Libraries like `clockwork` or `benbjohnson/clock` formalize this pattern.

Black-box vs white-box tests

Two package conventions:

- **White-box**: test file in the same package (`calc_test.go` declares `package calc`). Can access unexported names.
- **Black-box**: test file in a sibling package (`calc_test.go` declares `package calc_test`). Only sees the exported API. Catches API ergonomics issues.

You can mix both in the same directory. Use white-box for internal logic tests, black-box for API contract tests.

Golden files

For tests where the expected output is long (rendered HTML, formatted text, generated code). Store the expected output in a file. Compare during test. Regenerate with a flag.

```
var update = flag.Bool("update", false, "update golden files")

func TestRender(t *testing.T) {
    got := Render(input)
    golden := filepath.Join("testdata", "render.golden")

    if *update {
        os.WriteFile(golden, []byte(got), 0644)
    }

    want, err := os.ReadFile(golden)
    if err != nil {
        t.Fatal(err)
    }
    if got != string(want) {
        t.Errorf("output mismatch (use -update to regenerate)")
    }
}
```

`go test -update` regenerates the golden files. Review the diff before committing.

Best practices

- **Table-driven tests by default.** One test function, many cases. `t.Run` for naming.
- **Use `t.Helper()` in test helpers.** Failures point at the test, not the helper.
- **Use `t.Cleanup` for test setup.** Runs even after `t.Fatal`. Better than `defer` when registering cleanup from a helper.
- **Use `t.TempDir` and `t.Setenv`.** Auto-cleanup, no boilerplate.
- **Mark independent tests `t.Parallel()`.** Shrinks CI time.
- **Inject dependencies (clocks, stores, HTTP clients).** Don't call `time.Now()` or `http.Get` from tested code directly.
- **Test behavior at the boundaries.** Tests that target unexported internals break on every refactor.
- **Race detector in CI.** Always.
- **Treat coverage as a signal.** Strong assertions matter more than chasing 100%.
- **`go test -count=N`** to defeat the test result cache when you suspect flakiness.
- **Run benchmarks with `-count=10`** and `benchstat` for honest comparisons.

Common gotchas

- **Forgetting to capture loop variable in `t.Run` + `t.Parallel` (pre-Go 1.22).** Subtests run in parallel after the loop ends, all observing the same final `tc`. Fix with `tc := tc` or use Go 1.22+.
- **`t.Fatal` inside a goroutine.** `Fatal` calls `runtime.Goexit`, which only exits the calling goroutine. The test continues and may pass. Use `t.Error` + return from the goroutine, or signal back to the main test goroutine.
- **Tests cached even when you didn't expect.** Go caches test results based on inputs. Add `-count=1` to force a rerun.
- **Tests pass alone, fail in suite.** Shared state. Global variables, package-level caches, environment, files. Fix with `t.Cleanup` and `t.TempDir`.
- **Flaky tests due to time.** `time.Sleep(100ms)` then checking a result fails on slow CI. Inject a clock or use channels for synchronization.
- **`go test ./...` exits 0 when a package has no tests.** Doesn't mean tests passed. Means nothing ran. CI should still flag this when coverage is expected.
- **Forgetting to register cleanup for resources.** Leaked goroutines, open files, listening ports persist between tests.
- **Benchmark setup inside the loop.** Setup work counts toward the timing. Call `b.ResetTimer()` after setup.
- **Comparing maps and slices with `==`.** Compile error. Use `reflect.DeepEqual` or `slices.Equal` / `maps.Equal` (Go 1.21+).
- **`require` (or any `Fatal`) inside a parallel subtest.** Stops only that subtest. Siblings continue.
- **`testdata/` is a magic directory.** Go ignores it for compilation. Use it for fixtures.

Quick reference

Concept	Key fact
Test file	`*_test.go` in same or `_test` package
Test func	`func TestX(t *testing.T)`
Run	`go test ./...`
Filter	`go test -run TestX/case_name`
Helper	`t.Helper()` skips frame in failure traces
Subtest	`t.Run("name", func(t *testing.T) { ... })`
Parallel	`t.Parallel()` (must be safe to interleave)
Cleanup	`t.Cleanup(fn)` runs LIFO at test end
Temp dir	`t.TempDir()`
Env var	`t.Setenv(key, value)`
TestMain	One-time setup for the package
Benchmark	`func BenchmarkX(b *testing.B)` with `b.N` loop
`b.ResetTimer`	Exclude setup time
`-benchmem`	Report allocations
Example	`func ExampleX()` with `// Output:` comment
Fuzz	`func FuzzX(f *testing.F)` + `f.Fuzz(...)`
Coverage	`go test -cover` / `-coverprofile` / `-covermode=atomic`
Race detector	`go test -race ./...`
httptest server	`httptest.NewServer(handler)`
httptest recorder	`httptest.NewRecorder()`
`testdata/`	Fixture dir, ignored by build